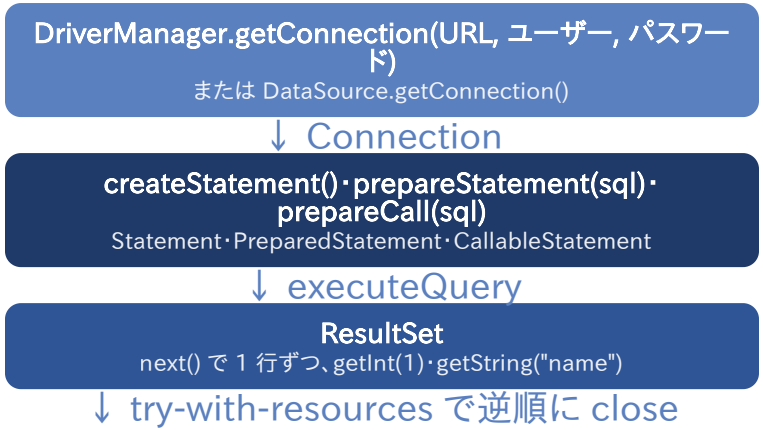
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 7 JDBC　#16

JDBC

DriverManager・DataSource、Statement・ResultSet、PreparedStatement・CallableStatement、トランザクション・バッチ

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の java.sql パッケージ（DriverManager・Connection・Statement・PreparedStatement・CallableStatement・ResultSet・Savepoint）と javax.sql.DataSource、The Java Tutorials の JDBC Database Access）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）と H2 Database 2.2.224 で実行して確かめた。例外のメッセージはドライバによってちがう

全体像 接続 → SQL → 結果 → 閉じる




```
String url = "jdbc:h2:mem:shop";
try (Connection c = DriverManager.getConnection(url, "sa", "");
     PreparedStatement ps = c.prepareStatement(
         "SELECT name FROM item WHERE price > ?")) {
  ps.setInt(1, 100);
  try (ResultSet rs = ps.executeQuery()) {
    while (rs.next()) System.out.println(rs.getString(1));
  }
}
```

Connection・Statement・ResultSet はすべてインタフェース（new できない）。
メソッドは SQLException（チェック例外）を投げる。

出典：API java.sql パッケージ・DriverManager・Connection／The Java Tutorials「Processing SQL Statements with JDBC」

第1章

接続

URL と DriverManager・DataSource。

JDBC URL と接続の取り方

jdbc	サブプロトコル	サブネーム
jdbc:	mysql:	//localhost:3306/shop
jdbc:	postgresql:	//host/db
jdbc:	h2:	mem:test

```
DriverManager.getConnection("jdbc:nodb:x");
// SQLException: No suitable driver found for jdbc:nodb:x

// throws も catch もない main で
Connection c = DriverManager.getConnection(url);
// × unreported exception SQLException
```

	DriverManager	DataSource
種類	クラス（java.sql）	インタフェース（javax.sql）
取り方	getConnection(URL, …)	getConnection()
接続先	コードに URL を書く	前もって設定（JNDI など）
接続プール	なし	組み合わせられる

> JDBC 4.0 からドライバは自動で登録される（Class.forName は不要）。

出典：API DriverManager・javax.sql.DataSource／JDK 17 と H2 でのコンパイルと実行の結果

第2章

Statement と ResultSet

3 つの execute と、カーソル。

3 つの execute

メソッド	戻り値	まちがえた SQL
executeQuery	ResultSet（0 行でも null でない）	DELETE など → SQLException
executeUpdate	int（行数、DDL は 0）	SELECT → SQLException
execute	boolean（ResultSet なら true）	なんでも可

> ResultSet r = s.executeUpdate(…); は「int cannot be converted to ResultSet」のコンパイルエラー。

```
s.executeUpdate("CREATE TABLE item(…)")       // 0
s.executeUpdate("INSERT … (1,'pen',100),
  (2,'ink',NULL),(3,'cup',300)")              // 3
s.executeUpdate("UPDATE item SET price =
  price + 1 WHERE price > 0")                 // 2（NULL は外れる）

s.execute("SELECT * FROM item")   // true
s.getUpdateCount()                // -1
s.execute("DELETE … id = 99")     // false
s.getUpdateCount()                // 0
```

出典：API Statement.executeQuery・executeUpdate・execute・getUpdateCount／JDK 17 と H2 での実行結果

ResultSet のカーソル

位置	id	name	price
（最初はここ）			
1 行目	1	pen	101
2 行目	2	ink	NULL
3 行目	3	cup	301

```
rs.getInt(1)      // next 前 → SQLException
while (rs.next()) { … }
rs.getInt(0)      // SQLException（列は 1 から）
rs.getString("ID") // 大文字・小文字は区別しない
```

```
// 2 行目（price が NULL）
int pr = rs.getInt("price"); // 0
rs.wasNull()                 // true
rs.getObject("price")        // null
rs.getString("name");
rs.wasNull()   // false（最後に読んだ name）

ResultSet r1 = s.executeQuery("SELECT 1");
ResultSet r2 = s.executeQuery("SELECT 2");
r1.isClosed()  // true（同じ Statement）
```

出典：API ResultSet・ResultSet.wasNull・Statement／JDK 17 と H2 での実行結果

スクロールとメタデータ

```
Statement sc = c.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_READ_ONLY);
ResultSet x = sc.executeQuery("SELECT id … ORDER BY id");
x.last();       x.getInt(1)  // 3（getRow も 3）
x.absolute(1);  x.getInt(1)  // 1
x.relative(1);  x.getInt(1)  // 2
x.absolute(-1); x.getInt(1)  // 3（最後）
x.absolute(10)               // false
x.beforeFirst(); x.next()    // true → 1
```

既定の TYPE_FORWARD_ONLY で previous は SQLException。

定数	意味
TYPE_FORWARD_ONLY	前に進むだけ（既定）
TYPE_SCROLL_INSENSITIVE	前後に動ける。ほかの変更を見ない
TYPE_SCROLL_SENSITIVE	前後に動ける。ほかの変更を見る
CONCUR_READ_ONLY・UPDATABLE	読むだけ・更新できる

```
ResultSetMetaData md = s.executeQuery(
  "SELECT id AS num, v FROM t").getMetaData();
md.getColumnCount()    // 2
md.getColumnLabel(1)   // NUM（別名）
md.getColumnName(1)    // ID（元の名前）
```

出典：API ResultSet.absolute・relative・last・previous、ResultSetMetaData／JDK 17 と H2 での実行結果

第3章

PreparedStatement と CallableStatement

? に値を入れる、プロシージャを呼ぶ。

PreparedStatement

```
PreparedStatement ps = c.prepareStatement(
  "SELECT name FROM item WHERE id = ? AND price > ?");
ps.setInt(1, 3);
ps.executeQuery();  // SQLException（2 つ目が未設定）
ps.setInt(2, 0);
ps.executeQuery();  // cup
ps.setInt(0, 1);    // SQLException（1 から）
ps.setInt(3, 1);    // SQLException（? は 2 つ）

PreparedStatement pu = c.prepareStatement(
  "UPDATE item SET name = ? WHERE id >= ?");
pu.setString(1, "x"); pu.setInt(2, 2);
pu.executeUpdate()  // 2
pu.setInt(2, 3);    // 1 つ目の値は残る
pu.executeUpdate()  // 1
```

Statement で文字列をつなぐ
"… WHERE name = '" + 入力 + "'" → SQL インジェクションの危険

PreparedStatement の ？
setString で値として入る → 防げる。前もって準備され、くり返しが速い

> executeQuery()・executeUpdate() は引数なし（SQL は作るときに渡した）。

出典：API PreparedStatement・clearParameters／The Java Tutorials「Using Prepared Statements」／JDK 17 と H2 での実行結果

CallableStatement

```
try (CallableStatement cs =
       c.prepareCall("{? = call TWICE(?)}")) {
  cs.registerOutParameter(1, Types.INTEGER);
  cs.setInt(2, 21);
  cs.execute();
  cs.getInt(1);   // 42
}
```

> CallableStatement は PreparedStatement の子。Statement → PreparedStatement → CallableStatement。

パラメータ	実行の前	実行のあと
IN	setXxx	—
OUT	registerOutParameter(番号, Types.〜)	getXxx
INOUT	setXxx と registerOutParameter	getXxx

```
{call 名前(?, ?)}          // プロシージャ
{? = call 名前(?)}         // 戻り値のある関数
```

出典：API CallableStatement・registerOutParameter・java.sql.Types／JDK 17 と H2 での実行結果

第4章

トランザクションとバッチ

まとめて確定、まとめて取り消し。

commit・rollback・Savepoint

```
// t は 1 行
c.getAutoCommit()            // true（既定）
c.setAutoCommit(false);
insert(2);
Savepoint sp = c.setSavepoint();
insert(3);
c.rollback(sp);   // 3 だけ取り消し → 2 行
c.commit();       // 2 行で確定
insert(4);
c.rollback();     // 4 を取り消し → 2 行
insert(5);
c.setAutoCommit(true);  // ここで commit
c.rollback();     // 戻らない → 3 行
```

自動コミット（既定）
SQL を 1 つ実行するたびに確定

setAutoCommit(false)
commit まで確定しない。rollback で最後の commit 以降を取り消す

Savepoint
rollback(sp) は印のあとだけ。トランザクションは続く

```
PreparedStatement b = c.prepareStatement(
  "INSERT INTO t VALUES (?, ?)");
for (…) { b.setInt(1, i); b.setInt(2, i); b.addBatch(); }
int[] res = b.executeBatch();  // [1, 1, 1]
```

出典：API Connection.setAutoCommit・commit・rollback・setSavepoint、Statement.addBatch・executeBatch／JDK 17 と H2 での実行結果

本番での問われ方

ねらい	見るところ
接続	jdbc:サブプロトコル:サブネーム、SQLException はチェック例外、DataSource はインタフェース
execute の使い分け	戻り値の型（ResultSet・int・boolean）、まちがえた SQL は実行時の SQLException
ResultSet	最初は 1 行目の前、列は 1 から、NULL の getInt は 0 と wasNull
PreparedStatement	? の番号は 1 から、未設定は例外、値は残る
CallableStatement	prepareCall、{call …}、registerOutParameter
トランザクション	既定は自動コミット、Savepoint、setAutoCommit(true) で commit

出典：試験内容チェックリスト 7／本資料の問題演習（全 30 問、JDK 17 と H2 で確認）